MOUSE SCRIPT INTERFACE

View online: https://www.construct.net/en/make-games/manuals/construct-3/scripting/scripting-reference/plugin-interfaces/mouse

---

The `IMouseObjectType` interface derives from IObjectClass to add APIs specific to the Mouse plugin.

Note this class derives from the object class interface, not the instance interface. Typically it is used through `runtime.mouse` instead of the named object.

Examples

The Shooting code example demonstrates pointing the player at the mouse cursor and shooting bullets towards it.

The Tracking pointers example also demonstrates how both mouse and touch input can be tracked simultaneously using pointer events.

Events

To detect mouse button events, use the events fired on the Runtime script interface, such as `"mousedown"` or `"pointerdown"`. Some additional Mouse-specific events are also available.

"movement"

Fired when the mouse cursor moves. The event object has properties `movementX` and `movementY` indicating the amount of movement. This is often used with pointer lock.

"pointerlockchange"

"pointerlockerror"

Fired when the state of pointer lock changes, or if an error occurs attempting to obtain pointer lock.

Mouse APIs

---

getMouseX(layerNameOrIndex)

getMouseY(layerNameOrIndex)

getMousePosition(layerNameOrIndex)

Return the current position of the mouse cursor on a layer, given by a case-insensitive string of its name or zero-based index. The parameter can be omitted for the default mouse

position, which does not take in to account any specific layer's transformations. `getMousePosition()` returns both the X and Y position as `[x, y]`.

---

isMouseButtonDown(button)

Return a boolean indicating if the given mouse button is currently down. The button is specified the same way as the MouseEvent.button property, i.e. 0 for left, 1 for middle, and 2 for right.

---

setCursorStyle(style)

Set the appearance of the mouse cursor to a string of a CSS `cursor` style value, e.g. `"crosshair"`. See cursor styles on MDN for some possible values.

---

setCursorObjectClass(objectClass)

Set the appearance of the mouse cursor to the current image of an IObjectClass. Various limitations apply: the object image is used as it appears in the image editor, not taking in to account size or rotation in the layout; the image cannot be too large (64x64 is usually the limit); the cursor may not be applied close to the edges of the browser window; and support varies depending on browser and OS. Some sample code for this method is shown below.

```
runtime.mouse.setCursorObjectClass(runtime.objects.Sprite);
```

---

hasPointerLock

A read-only boolean indicating if pointer lock is currently active.

---

requestPointerLock(opts)

Normally in DOM mode code can call the browser requestPointerLock method directly. However in worker mode and desktop exports, the browser method is unavailable or does not work. This method can be used to request pointer lock in these additional situations. Much like the browser API, `opts` is optional and can be an object with the property `unadjustedMovement` set to `true` to request that the system does not adjust the mouse movement values during pointer lock. The events `"pointerlockchange"` or `"pointerlockerror"` will be fired after this call depending on the outcome of the request.

---

releasePointerLock()

When pointer lock is active, release the pointer lock restoring normal mouse cursor operation.